DevOps Fundamentals

Compact Course — Day 2

CD Pipelines, Secrets, Docker & Observability

Day 2 — Agenda

Part 1 — Deployment Patterns

- Rolling, Blue/Green, Canary, and Recreate strategies
- Feature flags as a deployment tool
- Environment promotion and approval gates
- Rollback strategies

Part 2 — Secrets Management

- Why secrets in code is catastrophic
- GitHub and GitLab secret hierarchies
- OIDC and federated credentials — no long-lived secrets
- Secret scanning and pre-commit hooks

Part 3 — Docker in CI Pipelines

- Building, tagging, and pushing images in CI
- Container registries: GHCR and GitLab Registry
- Docker layer caching in pipelines
- Basic vulnerability scanning with Trivy

Part 4 — Monitoring and Observability

- The three pillars: Metrics, Logs, Traces
- Alerting philosophy and SLOs
- Pipeline observability in GitHub Actions and GitLab CI
- Status badges and step summaries

Part 5 — Advanced Patterns and GitOps

- DRY pipelines at scale
- Testing pyramid placement in CI
- Code quality gates
- Infrastructure pipelines (link to Terraform course)
- GitOps: pull vs. push deployment model

Part 1 — Deployment Patterns

Why Deployment Strategy Matters

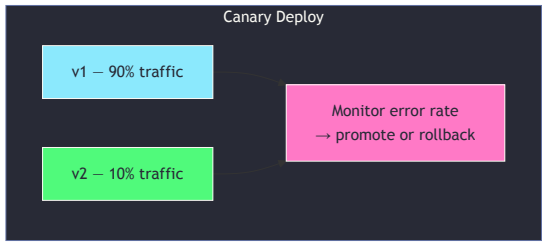
Deploying new code to production is inherently risky:

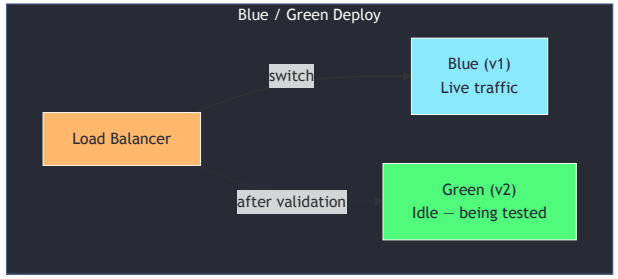
- New code may have bugs not caught in testing
- Deployment itself can cause downtime
- Rollback must be fast when things go wrong

The right strategy depends on:

- Acceptable **downtime window** (zero vs. scheduled)
- Required **rollback speed**
- Team **operational maturity**
- Application **architecture** (stateful vs. stateless)

Deployment Strategies







Rolling Deploy

Replace instances one at a time (or in batches):

```
Before:  [v1] [v1] [v1] [v1]
Step 1:  [v2] [v1] [v1] [v1]
Step 2:  [v2] [v2] [v1] [v1]
Step 3:  [v2] [v2] [v2] [v1]
After:   [v2] [v2] [v2] [v2]
```

Pros: No downtime, gradual rollout, low resource overhead

Cons: Briefly running mixed v1/v2 — DB schemas must be backwards compatible

Used by: Kubernetes Deployments (default strategy), ECS, Azure App Service slots

Blue / Green Deploy

Run two identical environments — switch traffic instantly:

```
Blue  (v1) ← live traffic
Green (v2) ← being prepared/tested

→ Switch load balancer → Green becomes live

Green (v2) ← now live
Blue  (v1) ← idle (instant rollback available)
```

Pros: Zero downtime, instant rollback (just flip back)

Cons: 2× infrastructure cost during deployment window

Used by: AWS CodeDeploy, Azure Deployment Slots, Spinnaker

Canary Deploy

Route a small percentage of traffic to the new version:

```
v1 — 95% of requests
v2 —  5% of requests  ← monitor for errors

→ promote: v1 — 50%, v2 — 50%
→ promote: v1 —  0%, v2 — 100%
```

Pros: Real-world validation with minimal blast radius

Cons: Requires traffic management (Nginx, Istio, feature flags)

Used by: Netflix, Google, Kubernetes with Argo Rollouts

Recreate Deploy

Stop all old instances, start all new ones:

```
[v1] [v1] [v1] [v1] → (down) → [v2] [v2] [v2] [v2]
```

Pros: Simple, no mixed versions

Cons: Downtime — only acceptable for non-production or batch systems

Feature Flags

Deploy code to production **without releasing it**:

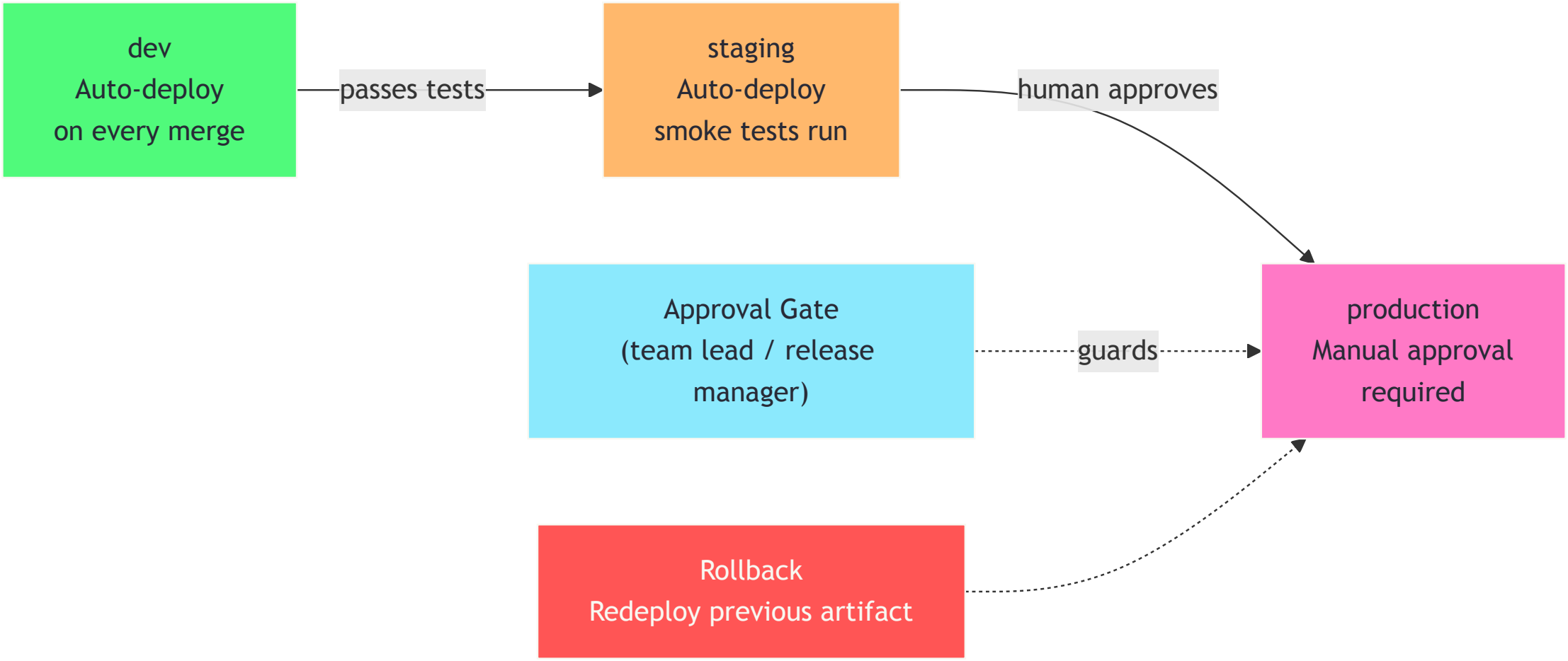
```
if (featureFlags.isEnabled('new-checkout', userId)) {
  // new checkout flow
} else {
  // old checkout flow
}
```

Benefits:

- Decouple deployment from release
- A/B testing and gradual rollouts
- Kill switch for broken features without redeploying
- Enables trunk-based development safely

Tools: LaunchDarkly, Flagsmith, Unleash, PostHog

Environment Promotion



Environment Promotion — Rules

Environment	Deploy trigger	Who approves	Rollback
`dev`	Every merge to main	Automatic	Re-run last pipeline
`staging`	Every successful dev deploy	Automatic	Re-run last pipeline
`production`	Every successful staging	Human approval	Rerun previous tag

GitHub Actions — environment protection rules:

```
Settings → Environments → production
✅ Required reviewers: [senior-engineer, team-lead]
✅ Wait timer: 0 minutes
✅ Deployment branches: main only
```

Rollback Strategies

Strategy	How	When to use
Rerun previous pipeline	Trigger the last known-good pipeline run	Blue/Green, artifact-based deploys
Git revert + pipeline	`git revert <bad-commit>` → CI → deploy	Any strategy
Blue/Green flip	Switch load balancer back to blue	Blue/Green only
Feature flag kill switch	Toggle flag off	Feature flag deploys
Helm rollback	`helm rollback <release> <revision>`	Kubernetes/Helm

Release vs. Deployment

These are often confused:

Term	Definition
Deployment	Putting new code onto servers — a technical operation
Release	Making a feature available to users — a business decision

Feature flags allow you to **separate these concerns**:

- Deploy code continuously throughout the day
- Release features on the business's schedule

Part 2 — Secrets Management

The Golden Rule

> **Never commit a secret to a Git repository — ever.**

What counts as a secret:

- API keys and tokens
- Database passwords and connection strings
- Private SSH keys and TLS certificates
- Cloud provider credentials (AWS access keys, Azure SP secrets)
- Webhook secrets

Git history is **permanent**. Even if you delete the file, the secret is in every clone.

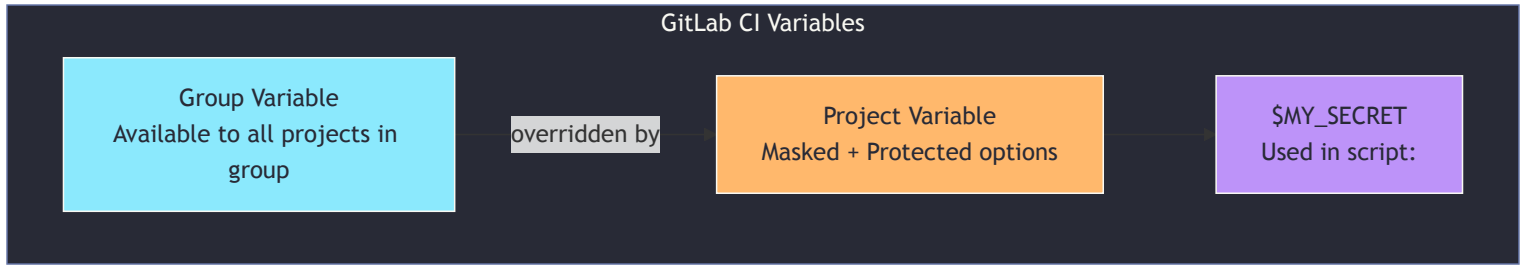
What Happens When Secrets Leak

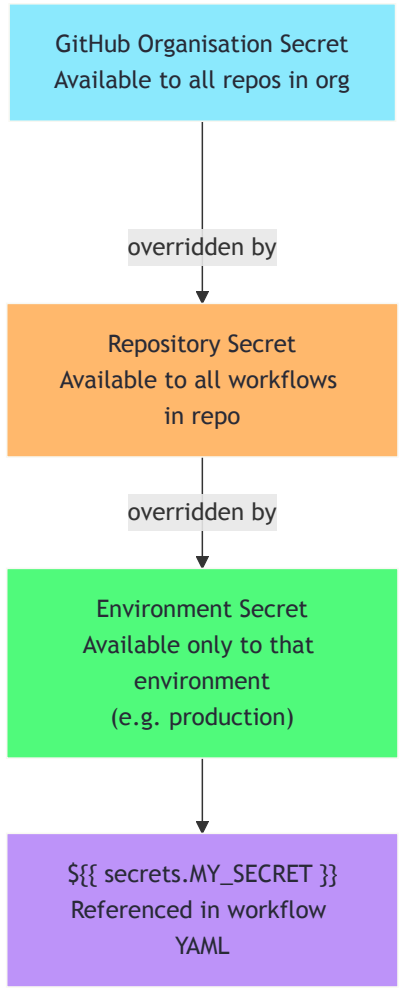
Real-world consequences:

- **AWS keys on GitHub** → attacker spins up crypto mining at your cost within seconds (automated scanners watch GitHub 24/7)
- **Database password in repo** → data breach, regulatory fines
- **Service account key** → full access to your cloud environment
- **Webhook secret** → attacker can forge events and trigger pipelines

> GitHub Secret Scanning alerts you within minutes of a push. But by then it may already be too late.

GitHub Secrets Hierarchy





GitHub Secrets — Practical Setup

```yaml
# Organisation secret  → available to all repos in the org
# Repository secret    → available to all workflows in this repo
# Environment secret   → available only when targeting that environment

jobs:
  deploy:
    environment: production          # activates environment secrets
    runs-on: ubuntu-latest
    steps:
      - name: Deploy
        env:
          DB_PASSWORD: ${{ secrets.DB_PASSWORD }}   # env secret
          API_KEY: ${{ secrets.API_KEY }}            # repo secret
          ORG_TOKEN: ${{ secrets.ORG_SHARED_TOKEN }} # org secret
        run: ./deploy.sh
```

Environment secrets override repository secrets of the same name.

GitLab CI Variables — Masked and Protected

```yaml
deploy:
  script:
    - echo "Token is $DEPLOY_TOKEN"   # shows as [MASKED] in logs
```

Setting variables in GitLab UI:

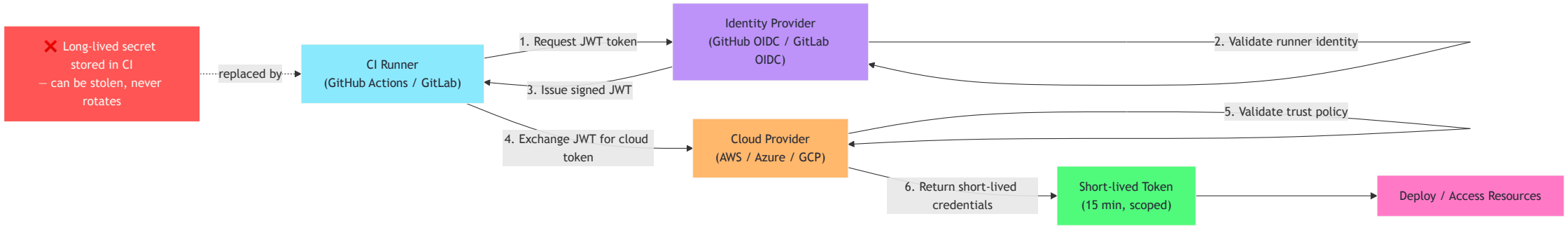
```
Settings → CI/CD → Variables → Add variable

Key:       DEPLOY_TOKEN
Value:     secret-value-here
Type:      Variable
Protected: ✅  (only available on protected branches/tags)
Masked:    ✅  (never shown in job logs)
```

File type variables — for SSH keys, certificates:

```
Type: File  → GitLab writes it to a temp file, sets var to the path
```

OIDC — No Long-Lived Credentials



OIDC with GitHub Actions + AWS

```yaml
permissions:
  id-token: write   # required for OIDC
  contents: read

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/github-actions
          aws-region: eu-west-1
          # No access key or secret key — pure OIDC
      - run: aws s3 ls
```

AWS IAM trust policy (configured once):

```
{
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:sub":
        "repo:org/repo:ref:refs/heads/main"
    }
  }
}
```

Secret Scanning

GitHub Advanced Security (free for public repos):

- Scans every push for 200+ secret patterns
- Alerts repository admins immediately
- Push protection: blocks pushes containing secrets

GitLab Secret Detection:

```
include:
  - template: Security/Secret-Detection.gitlab-ci.yml
```

Pre-commit hooks (shift left — catch before push):

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.4.0
    hooks:
      - id: detect-secrets
```

Install: `pip install pre-commit && pre-commit install`

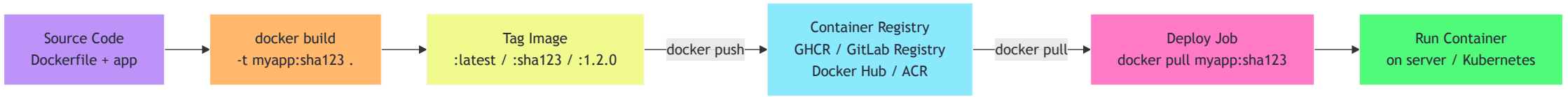
Practice	Why
Rotate secrets regularly	Limits blast radius of a leak
Use OIDC instead of static credentials	Short-lived, auto-rotates
Principle of least privilege	Secrets only have access they need
Separate secrets per environment	Prod breach does not affect dev
Audit secret access	Know who used what, when
Use a secrets manager for complex setups	HashiCorp Vault, AWS Secrets Manager, Azure Key Vault
Never log secrets	Use `::add-mask::` in GitHub Actions if dynamic

Part 3 — Docker in CI Pipelines

Docker Basics Recap

Concept	Description
Image	Read-only template — layers of filesystem changes
Container	Running instance of an image
Dockerfile	Instructions to build an image
Registry	Storage for images (Docker Hub, GHCR, GitLab Registry, ACR)
Tag	Label for a specific image version (`:latest`, `:1.2.0`, `:abc1234`)

Docker in CI


Source Code
Dockerfile + app
docker build
-t myapp:sha123 .
Tag Image
:latest / :sha123 / :1.2.0
docker push
Container Registry
GHCR / GitLab Registry
Docker Hub / ACR
docker pull
Deploy Job
docker pull myapp:sha123
Run Container
on server / Kubernetes

A Simple Dockerfile

```
# Build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Runtime stage (smaller image)
FROM node:20-alpine AS runtime
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/index.js"]
```

Multi-stage builds keep the final image small — no dev dependencies, no source code.

Building in GitHub Actions

```yaml
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write   # required to push to GHCR

    steps:
      - uses: actions/checkout@v4

      - name: Log in to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}   # built-in, no setup needed

      - name: Build and push
        uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: |
            ghcr.io/${{ github.repository }}:latest
            ghcr.io/${{ github.repository }}:${{ github.sha }}
```

Tagging Strategy

Tag	When to use	Notes
`:latest`	Current main branch build	Mutable — always points to newest
`:abc1234` (commit SHA)	Every build	Immutable — exact reproducibility
`:1.2.0` (SemVer)	Tagged releases	Immutable — prefer for production
`:main` or `:develop`	Branch tracking	Useful for staging environments

Best practice:

- Never deploy `:latest` to production
- Always deploy a specific SHA or version tag
- Keep `:latest` for development/testing convenience

Docker Layer Caching in CI

```yaml
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Build and push with cache
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ghcr.io/${{ github.repository }}:${{ github.sha }}
    cache-from: type=gha            # restore from GitHub Actions cache
    cache-to: type=gha,mode=max     # save to GitHub Actions cache
```

How it works:

- Docker BuildKit saves individual layer digests
- Unchanged layers are restored from cache — no rebuild
- `COPY package*.json` + `RUN npm ci` only re-runs when `package.json` changes

Building in GitLab CI

```
build-docker:
  stage: build
  image: docker:24
  services:
    - docker:24-dind              # Docker-in-Docker
  variables:
    DOCKER_TLS_CERTDIR: '/certs'
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```

GitLab provides predefined registry variables:

- `$CI_REGISTRY` — `registry.gitlab.com`
- `$CI_REGISTRY_IMAGE` — `registry.gitlab.com/group/project`
- `$CI_REGISTRY_USER` / `$CI_REGISTRY_PASSWORD` — auto-generated per job

Vulnerability Scanning with Trivy

Add to your pipeline after the build step:

```yaml
- name: Scan image with Trivy
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: ghcr.io/${{ github.repository }}:${{ github.sha }}
    format: table
    exit-code: '1'                # fail the pipeline on findings
    severity: 'HIGH,CRITICAL'     # only fail on HIGH and CRITICAL
    ignore-unfixed: true          # skip if no fix is available yet
```

GitLab — built-in container scanning:

```yaml
include:
  - template: Security/Container-Scanning.gitlab-ci.yml

container_scanning:
  variables:
    CS_IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
```

Part 4 — Monitoring and Observability

Monitoring vs. Observability

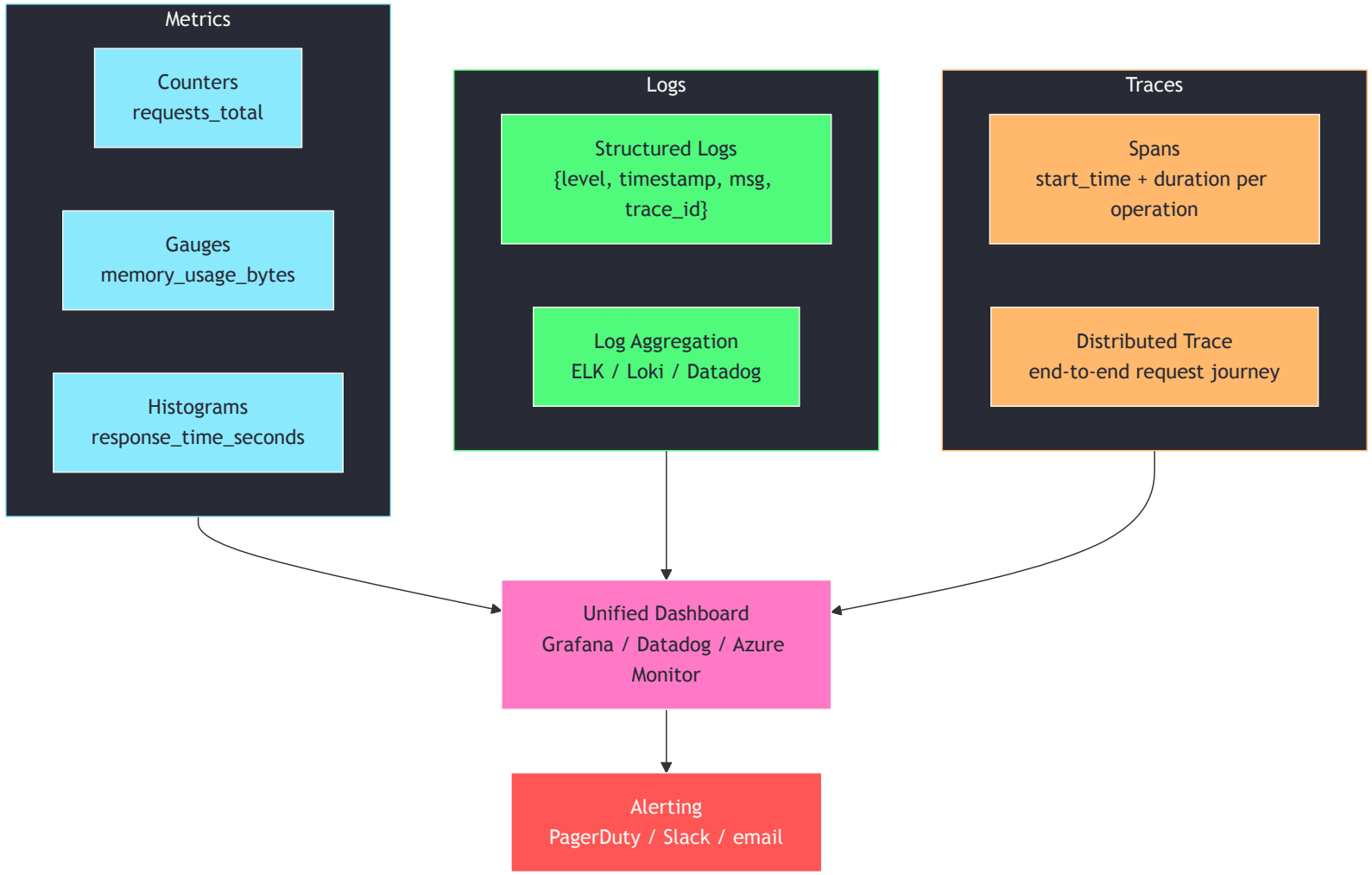
Monitoring: Watching known metrics — "is this value above threshold?"

Observability: Understanding unknown-unknowns — "why is this behaving this way?"

An observable system lets you ask **new questions** about production behaviour without deploying new instrumentation.

The three pillars of observability:

The Three Pillars



Metrics

Time-series numeric data:

Type	Example	Use
Counter	`http_requests_total`	Always increasing — rate of change matters
Gauge	`memory_usage_bytes`	Point-in-time value (can go up or down)
Histogram	`request_duration_seconds`	Distribution of values — p50, p99 latency

Prometheus model:

```
http_requests_total{method="GET", status="200", path="/api/users"} 1542
```

Labels make metrics multi-dimensional — slice by any combination.

Logs

Structured logs are far more useful than plain text:

```
{
  "timestamp": "2026-07-02T14:32:01Z",
  "level": "error",
  "message": "Payment processing failed",
  "trace_id": "abc-123-xyz",
  "user_id": "usr-789",
  "amount": 99.99,
  "error": "upstream timeout after 30s"
}
```

Why structured?

- Machine-parseable → queryable in Elasticsearch / Loki / Splunk
- `trace_id` correlates with distributed traces
- Filter and aggregate across millions of log lines

Traces

Distributed tracing follows a request across multiple services:

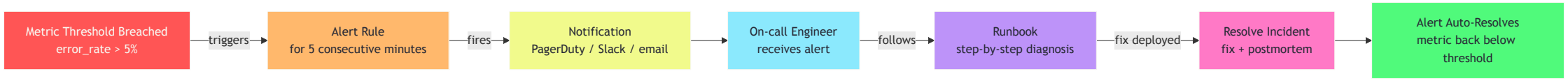
```
Request: POST /checkout (100ms total)
  ├── auth-service.validateToken   (5ms)
  ├── cart-service.getCart         (8ms)
  ├── inventory-service.reserve    (12ms)
  └── payment-service.charge       (72ms)  ← bottleneck
        └── external-gateway.call  (68ms)  ← root cause
```

How it works:

- Each service propagates a `trace_id` in request headers
- Each operation becomes a `span` with start time + duration
- Spans are stitched together by the `trace_id`

Tools: Jaeger, Zipkin, OpenTelemetry, Datadog APM

Alerting


Metric Threshold Breached
error_rate > 5%
triggers
Alert Rule
for 5 consecutive minutes
fires
Notification
PagerDuty / Slack / email
On-call Engineer
receives alert
follows
Runbook
step-by-step diagnosis
fix deployed
Resolve Incident
fix + postmortem
Alert Auto-Resolves
metric back below
threshold

Alerting — Philosophy

Principle	Description
Actionable	Every alert must have a clear action. If you can't do anything, it's not an alert — it's noise
Symptom-based	Alert on user-visible symptoms (`error_rate > 1%`) not causes (`cpu_usage > 80%`)
Low noise	Alert fatigue causes engineers to ignore all alerts. Tune ruthlessly
Linked to runbook	Every alert links to step-by-step diagnosis documentation
Severity levels	`critical` (wake someone up), `warning` (business hours), `info` (log only)

SLOs and SLAs

SLA (Service Level Agreement): External contract — "we guarantee 99.9% uptime"

SLO (Service Level Objective): Internal target — "we aim for 99.95% uptime" (tighter than SLA)

SLI (Service Level Indicator): The metric you measure — "actual uptime over 30 days"

Error budget:

```
SLO: 99.9% availability = 0.1% error budget
In 30 days = 43.2 minutes of allowed downtime

Spent so far: 31 minutes
Remaining:    12.2 minutes
```

When the error budget is exhausted: **freeze feature releases, focus on reliability**.

Pipeline Observability — GitHub Actions

```yaml
steps:
  - name: Run tests
    run: npm test

  # Write a rich job summary (shown in GitHub Actions UI)
  - name: Write summary
    if: always()
    run: |
      echo "## Test Results" >> $GITHUB_STEP_SUMMARY
      echo "| Status | Count |" >> $GITHUB_STEP_SUMMARY
      echo "|--------|-------|" >> $GITHUB_STEP_SUMMARY
      echo "| Passed | $(cat results.json | jq .passed) |" >> $GITHUB_STEP_SUMMARY
      echo "| Failed | $(cat results.json | jq .failed) |" >> $GITHUB_STEP_SUMMARY
```

Key metrics to track:

- Pipeline success rate
- Average job duration over time
- Flaky tests (passes sometimes, fails others)
- Cache hit rate
- Runner queue time

Pipeline Observability — GitLab CI

GitLab provides built-in **Pipeline Analytics**:

```
CI/CD → Pipelines → Charts
  – Success rate over time
  – Pipeline duration trends
  – Individual job duration breakdown
```

Failure notifications:

```
notify-failure:
  stage: .post
  script:
    - |
      curl -X POST "$SLACK_WEBHOOK_URL" \
        -H 'Content-type: application/json' \
        --data "{\"text\": \"❌ Pipeline failed on $CI_COMMIT_REF_NAME - $CI_PIPELINE_URL\"}"
  when: on_failure
```

Status Badges

Add to your `README.md`:

GitHub Actions:

```
![CI](https://github.com/org/repo/actions/workflows/ci.yml/badge.svg)
```

GitLab CI:

```
[![pipeline status](https://gitlab.com/org/repo/badges/main/pipeline.svg)](https://gitlab.com/org/repo/-/commits/main)
[![coverage report](https://gitlab.com/org/repo/badges/main/coverage.svg)](https://gitlab.com/org/repo/-/commits/main)
```

Result: a live badge on your README showing current pipeline status.

Part 5 — Advanced Patterns and GitOps

DRY Pipelines at Scale

GitHub Actions — composite actions:

```yaml
# .github/actions/setup-node/action.yml
name: Setup Node
runs:
  using: composite
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
    - uses: actions/cache@v4
      with:
        path: ~/.npm
        key: ${{ runner.os }}-npm-${{ hashFiles('**/package-lock.json') }}
```

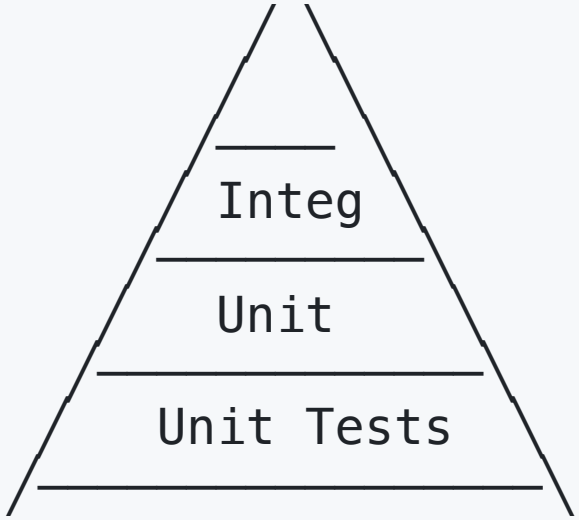
GitLab CI — anchors:

```yaml
.common-before: &common-before
  before_script:
    - npm ci

test:
  <<: *common-before
  script: npm test

lint:
  <<: *common-before
  script: npm run lint
```

The Testing Pyramid in CI




```
  E2E Tests (few, slow, brittle)
  → run nightly or on main only

Integration Tests (some, moderate speed)
 → run on every PR

Unit Tests (many, fast, reliable)
 → run on every commit
```

Placement in CI:

Test type	When to run	Fail behaviour
Unit	Every commit	Block merge
Integration	Every PR	Block merge
E2E	Post-merge / nightly	Alert, do not block
Load / performance	Weekly / pre-release	Alert, do not block

Code Quality Gates

```
# GitHub Actions
- name: Lint
  run: npm run lint

- name: Type check
  run: npx tsc --noEmit

- name: Coverage threshold
  run: npm test -- --coverage --coverageThreshold='{"global":{"lines":80}}'

- name: Security audit
  run: npm audit --audit-level=high
```

GitLab CI — Code Quality report:

```
include:
  - template: Code-Quality.gitlab-ci.yml
```

Appears as a diff in Merge Request — shows new issues introduced by the MR.

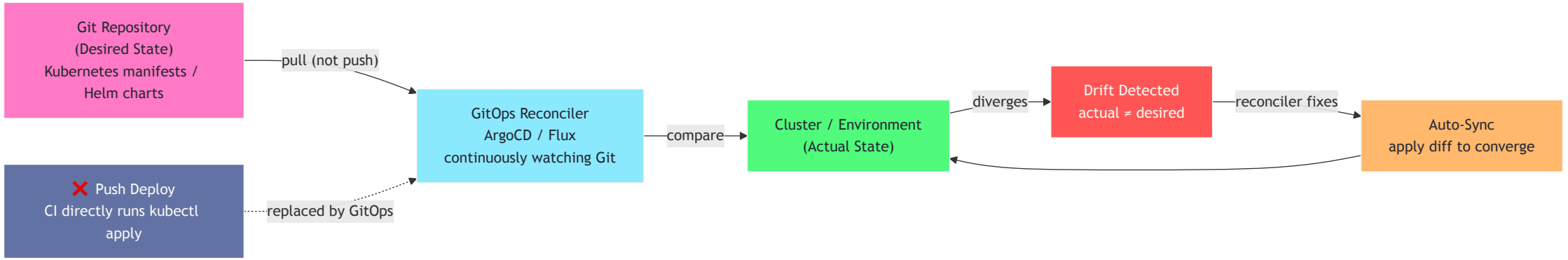
Infrastructure Pipelines

CI/CD is not only for application code — run Terraform through pipelines too:

```yaml
# .github/workflows/terraform.yml
jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
      - run: terraform plan -out=tfplan
      - uses: actions/upload-artifact@v4
        with:
          name: tfplan
          path: tfplan

  apply:
    needs: plan
    environment: production   # requires approval
    steps:
      - uses: actions/download-artifact@v4
        with: { name: tfplan }
      - run: terraform apply tfplan
```

GitOps


Git Repository
(Desired State)
Kubernetes manifests /
Helm charts
pull (not push)
GitOps Reconciler
ArgoCD / Flux
continuously watching Git
compare
Cluster / Environment
(Actual State)
diverges
Drift Detected
actual ≠ desired
reconciler fixes
Auto-Sync
apply diff to converge
❌ Push Deploy
CI directly runs kubectl
apply
replaced by GitOps

GitOps Principles

1. **Declarative:** The entire system is described declaratively (Kubernetes manifests, Helm charts)
2. **Versioned and immutable:** Desired state stored in Git — every change is a commit
3. **Pulled automatically:** Approved changes are automatically applied by a reconciler
4. **Continuously reconciled:** Running system is continuously observed and corrected

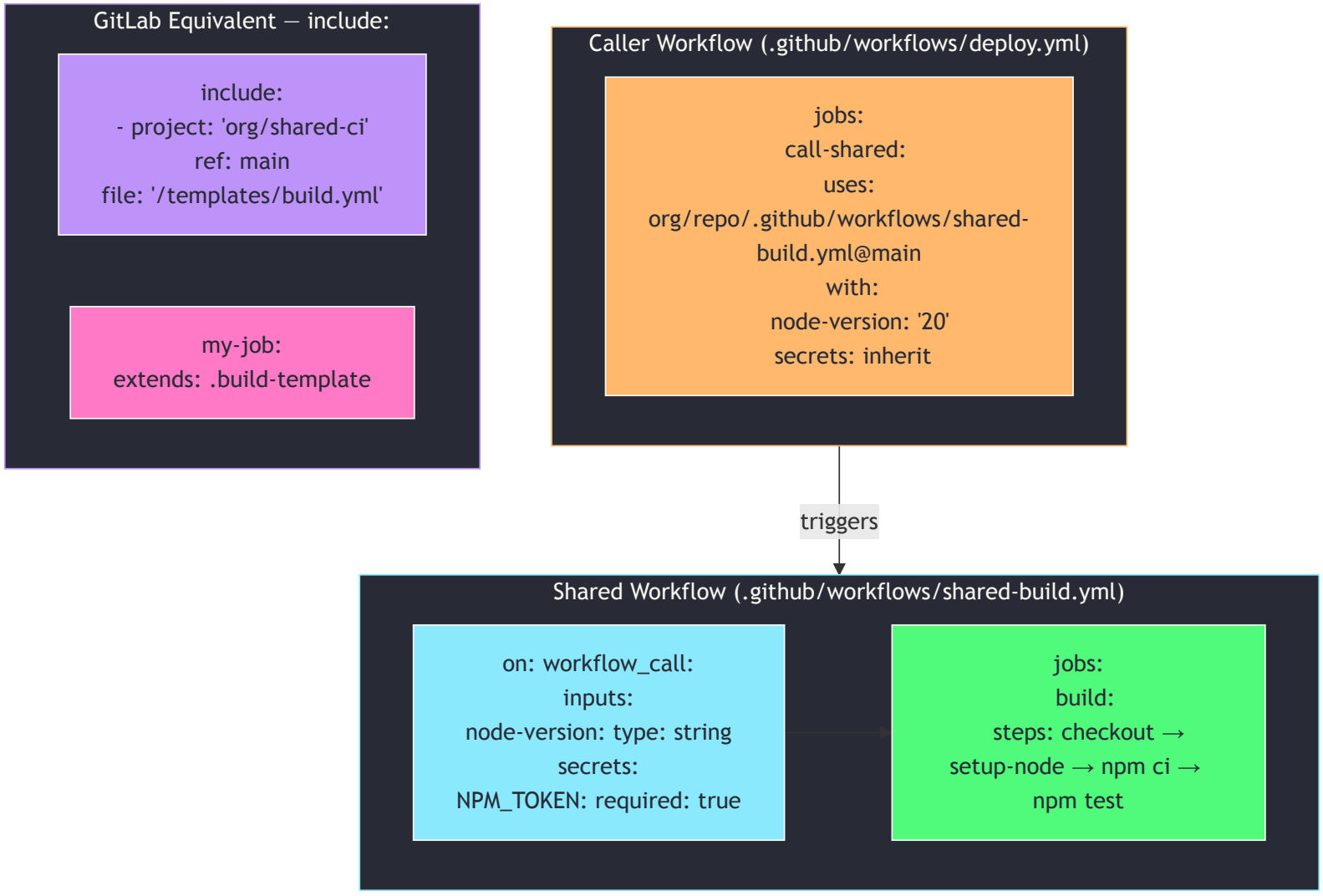
Tools:

Tool	Type	Notes
ArgoCD	Kubernetes-native	UI + CLI, multi-cluster, RBAC
Flux	Kubernetes-native	Lightweight, Helm + Kustomize
GitHub Actions deploy	Push model	Simpler but not true GitOps

Pull vs. Push Deployment

	Push (classic CI/CD)	Pull (GitOps)
Who deploys	CI runner pushes to environment	Reconciler in environment pulls from Git
Credentials	CI runner needs deploy credentials	Cluster only needs Git read access
Drift correction	Manual — next deploy fixes it	Automatic — reconciler fixes drift continuously
Auditability	Pipeline logs	Git commits are the audit trail
Rollback	Re-run old pipeline	`git revert` → auto-applied
Complexity	Lower	Higher (need reconciler running)

Reusable Workflows



Day 2 — Summary

What we covered:

1. **Deployment patterns** — Rolling, Blue/Green, Canary, feature flags, environment promotion, rollback
2. **Secrets management** — GitHub/GitLab hierarchies, OIDC federated credentials, secret scanning, pre-commit hooks
3. **Docker in CI** — Dockerfile, build/tag/push to GHCR and GitLab Registry, layer caching, Trivy scanning
4. **Monitoring and observability** — metrics, logs, traces, alerting philosophy, SLOs, pipeline analytics, status badges
5. **Advanced patterns and GitOps** — DRY pipelines, testing pyramid, code quality gates, infrastructure pipelines, GitOps pull model

Course Summary — DevOps Fundamentals

Day 1 — Foundation:

- DevOps culture, Three Ways, DORA metrics
- Git branching strategies (GitFlow, GitHub Flow, GitLab Flow)
- CI/CD fundamentals, GitHub Actions, GitLab CI

Day 2 — Production:

- CD pipelines, deployment strategies, approval gates
- Secrets management and OIDC
- Docker in CI, vulnerability scanning
- Observability, alerting, SLOs, GitOps

What comes next?

- **Terraform with Azure** — Infrastructure as Code, the other half of the DevOps toolchain
- **Kubernetes / AKS** — Container orchestration for production workloads
- **Platform Engineering** — Internal developer platforms, golden paths